



Práctica 3

Objetivo

El alumno se familiarizará con el uso del periférico UART usando el sistema embebido ESP32 DevKit v1 para desarrollar aplicaciones para sistemas basados en microcontrolador para aplicarlos en la resolución de problemas de cómputo, de una manera eficaz y responsable

Equipo

Computadora personal con conexión a Internet.

Teoría

- Describa el modo UART half-duplex del ESP32.

Desarrollo

Parte 1.

Cree una biblioteca de funciones para el ESP32 ESP-IDF donde implemente las siguientes funciones:

- **char uart_getchar(uart_port_t uart_num):** Recibe un caracter del puerto UART indicado y lo retorna.
- **void uart_gets(uart_port_t uart_num, char * str, uint8_t max_len):** Recibe una cadena de caracteres del puerto UART indicado y la almacena en **str**. La longitud máxima es **max_len** incluyendo el fin de cadena.
- **void uart_getNum(uart_port_t uart_num, char * str, uint8_t max_len):** Recibe una cadena de caracteres validada a solo **caracteres numéricos**, del puerto UART indicado y la almacena en **str**. La longitud máxima es **max_len** incluyendo el fin de cadena.
- **void uart_getAlpha(uart_port_t uart_num, char * str, uint8_t max_len):** Recibe una cadena de caracteres validada a solo **letras**, del puerto UART indicado y la almacena en **str**. La longitud máxima es **max_len** incluyendo el fin de cadena.
- **void putchar(uart_port_t uart_num, char c):** Escribe un caracter en el puerto UART indicado.
- **void puts(uart_port_t uart_num, char * str):** Escribe una cadena de caracteres en el puerto UART indicado.

Parte 2.

Implemente en dos ESP32 ESP-IDF una pancarta ASCII haciendo uso de **UART, tareas y mecanismos de sincronización de tareas**. La implementación debe ser eficiente en el uso de recursos de cómputo (procesador, memoria y periféricos). La implementación se realiza en los **boards ESP32**, no en el simulador.

Un ESP32 recibe del usuario una cadena de texto de máximo 25 caracteres usando UART. El ESP32 envía por UART la cadena al otro ESP32. Este ESP32 convierte los caracteres a un formato de pancarta ASCII y los despliega en la terminal por medio de UART. Limite los caracteres válidos a las letras del alfabeto en mayúscula y minúscula, números, espacio y los caracteres especiales !, ., + y -. La recepción de la cadena del usuario debe estar validada y debe aceptar las teclas Backspace y Enter.

Fig. 1. Diagrama a bloques.



La comunicación entre los dos ESP32 utiliza una trama con el siguiente formato:

<SF>MENSAJE<EF><CHECKSUM>

Donde:

Campo	Descripción
SF	Carácter de inicio de trama. Use el caracter de su elección.
MENSAJE	Texto enviado por el usuario.
EF	Carácter de fin de trama. Use el caracter de su elección.
CHECKSUM	Valor para verificar la integridad del mensaje.

Cálculo del checksum

El checksum se calcula mediante una operación XOR de todos los caracteres del mensaje.

Cuando el ESP32 2 recibe la trama, realizar el cálculo del checksum del mensaje y lo compara con el checksum que recibió en la trama. Si el checksum es correcto (el que calculó y el que recibió son iguales), responde al ESP32 1: **ACK**. Si el checksum es incorrecto (el que calculó y el que recibió son diferentes), responde al ESP32 1: **NACK**.

El ESP32 2 también responde **NACK** si el formato de la trama es incorrecto.

El ESP32 1 al recibir **ACK** despliega en la terminal: Mensaje enviado correctamente. Al recibir **NACK** despliega en la terminal: Error al enviar mensaje.

El ESP32 2 únicamente despliega la pancarta ASCII si recibió el mensaje correctamente, es decir, el formato de la trama recibida y su checksum fueron correctos.

Ejemplo 1 – Mensaje enviado correctamente:

Si SF = '#' y EF = '\$'

El usuario escribe: Hola Mundo!

El ESP32 1 transmite:

#Ho la Mundo!\$[CHECKSUM = 0x76]

0x23	0x48	0x6F	0x6C	0x61	0x20	0x4D	0x75	0x6E	0x64	0x6F	0x21	0x24	0x76
------	------	------	------	------	------	------	------	------	------	------	------	------	------

El ESP32 1 despliega en la terminal: Mensaje enviado correctamente.

El ESP32 2 despliega en la terminal la pancarta ASCII.

Ejemplo 2 – Mensaje enviado con error en checksum:

Si SF = '#' y EF = '\$'

El usuario escribe: Hola Mundo!

El ESP32 1 transmite:

#Ho la Mundo!\$[CHECKSUM = 0x70]

0x23	0x48	0x6F	0x6C	0x61	0x20	0x4D	0x75	0x6E	0x64	0x6F	0x21	0x24	0x70
------	------	------	------	------	------	------	------	------	------	------	------	------	------

El ESP32 1 despliega en la terminal: Error al enviar mensaje.

El ESP32 2 no despliega en la terminal.

La pancarta ASCII es de su elección pero debe ser similar a las de los ejemplos en las Fig. 2 y 3 y debe ser legible.

Haga uso de las funciones de UART.h para recibir, enviar y desplegar los caracteres. Es decir, use `uart_read_bytes`, `uart_write_bytes` y demás funciones de UART.h. No use `scanf`, `printf`, etc.

El sistema debe tener por los menos dos tareas en cada ESP32:

ESP32 1:

- **Tarea de recepción de mensaje:** Lee caracteres de la terminal, valida los caracteres y la longitud, detecta Enter.
- **Tarea de transmisión de trama:** Espera el mensaje de la tarea anterior, calcula el checksum, contruye la trama, envía la trama al ESP32 2 y espera ACK/NACK.

ESP32 2:

- **Tarea de recepción de trama:** Recibe la trama y verifica que tenga el formato correcto.
- **Tarea de transmisión de pancarta:** Recibe la trama de la tarea anterior, verifica que tenga el checksum correcto, genera respuesta ACK/NACK, genera pancarta ASCII y la muestra en la terminal.

Puede agregar más tareas según lo considere necesario.

En el reporte incluya:

- Una descripción de las tareas que implementó y la función de cada tarea.
- Una descripción de los mecanismos de colas, Mutex, semáforos o Event groups que usó indicando por qué y para qué los uso.

Fig. 2. Ejemplo 1 de pancarta.



Fig. 3. Ejemplo 2 de pancarta.



Conclusiones y comentarios

Dificultades en el desarrollo

Referencias